Terminus CertSim Code Review

Oct 20, 2026 · @CJ

Summary

The project is in good shape: it installs, typechecks, builds and passes all 9 tests, but 4 bugs in chmod and redirection would teach wrong exam answers. This review covers cjmit18/terminalpractice at commit `bc164ee` ("implement Linux/Windows command chaining", pushed Oct 20, 2026), about 8,300 lines of React and TypeScript.

Check	Command	Result
Clean install	`npm ci`	Pass, 61 packages
Typecheck	`npm run lint`	Pass, 0 errors (was 15)
Build	`npm run build`	Pass, 441 KB JS (126 KB gzip)
Tests	`npm test`	Pass, 9 of 9

Every finding below was reproduced by running commands through the engines directly, not just by reading code.

Fixed since last review

All 6 issues from the Oct 19 review were addressed in one commit.

- [x] ~~**Install fails.** Removed the conflicting `esbuild ^0.25` pin and the unused `@google/genai`, `express`, `dotenv` and `motion`. Lockfile now committed.~~
- [x] ~~**15 type errors.** Annotated `baseLinuxState` and `baseWindowsState` as `SystemState`, and fixed the `boolean | undefined` verifier.~~
- [x] ~~**Wrong Permission Matrix presets.** `applyPreset` now parses a preset string, so 4755 and 1777 display correctly. Initial state is also derived from `initialOctal`.~~
- [x] ~~**No permission enforcement.** New `checkNodePermission`, `canReadNode`, `canWriteNode` and `canDeleteNode` cover read, write, traversal, ownership and the sticky bit. `sudo`, `su` and `exit` change the effective user, and `systemctl`, `apt` and `fdisk` require root.~~

- [x] ~~**Fragile parsing.** Quote aware splitting for `&&`, `||` and `;`; quote aware redirects; grep with no match now exits 1 silently.~~
- [x] ~~**Cleanup.** Package renamed to `terminus-certsim`, `test` script added, `.aistudio/` ignored and its 1.2 MB of screenshots removed.~~

Findings and solutions

10 findings remain. High means the sim teaches a wrong exam answer; Medium means a real command behaves wrongly; Low is polish.

#	Finding	Severity	File
1	chmod drops SUID, SGID and sticky bits	High	linuxEngine.ts
2	Symbolic chmod mostly unimplemented	High	linuxEngine.ts
3	`sudo echo x > /root/file` succeeds	High	linuxEngine.ts
4	stderr redirects (`2>` , `2>>`) broken	Medium	parser.ts
5	`mkdir -p` fails with a false "Permission denied"	Medium	filesystem.ts, linuxEngine.ts
6	Errors from earlier chained commands are lost	Medium	linuxEngine.ts, windowsEngine.ts
7	CMD uses `;` instead of `&`, and CMD pipes are ignored	Medium	parser.ts, windowsEngine.ts
8	`rm` and `mkdir` edge cases	Low	linuxEngine.ts
9	Supplementary groups ignored in permission checks	Low	filesystem.ts
10	Windows redirect files created as mode 777	Low	windowsEngine.ts

1 and 2. chmod special bits and symbolic modes

Evidence. `chmod 4755 t.sh` gives `rwxr-xr-x`, because line 556 parses `modeArg.slice(-3)`. `chmod 1777` gives `rwxrwxrwx`. `chmod u+x,g-r` and `chmod o=r` change nothing but exit 0. The `else if (modeArg === '750')` branch is unreachable,

since the octal regex above it already matches 750. The module's own exam tips teach SUID 4000 and sticky 1000, so the terminal contradicts the lesson.

Solution. Add one `applyChmod` helper to `filesystem.ts` that handles 3 or 4 digit octal and full symbolic syntax (`[ugoa][+-=][rwxst]`, comma lists). Tested against 13 cases, including 4755, 2755, 1777, 1666 (`T`), `g-w`, `a+x`, `o=r`, `u+s`, `u-s`, `+t` and `u+x,g-r,o-rx`; all pass.

```
export function applySpecialBits(sym: string, special: number): string {
  const c = sym.split('');
  if (special & 4) c[2] = c[2] === 'x' ? 's' : 'S';
  if (special & 2) c[5] = c[5] === 'x' ? 's' : 'S';
  if (special & 1) c[8] = c[8] === 'x' ? 't' : 'T';
  return c.join('');
}

export function getSpecialBits(sym: string): number {
  return (/[sS]/.test(sym[2]) ? 4 : 0) | (/[sS]/.test(sym[5]) ? 2 : 0) |
(/[tT]/.test(sym[8]) ? 1 : 0);
}

export function applyChmod(current: string, mode: string): { sym: string;
octal: number } | null {
  let special = getSpecialBits(current);
  let digits = String(symbolicToOctal(current)).padStart(3,
'0').split('').map(Number);

  if (/^[0-7]{3,4}$/.test(mode)) {
    const m = mode.padStart(4, '0');
    special = Number(m[0]);
    digits = m.slice(1).split('').map(Number);
  } else {
    for (const clause of mode.split(',')) {
      const match = clause.match(/^([ugoa]*)([+\-=])([rwxst]*)$/);
      if (!match) return null;
      const [, whoRaw, op, perms] = match;
      const who = whoRaw === '' || whoRaw.includes('a') ? 'ugo' : whoRaw;
      const rwx = (perms.includes('r') ? 4 : 0) | (perms.includes('w') ? 2 :
0) | (perms.includes('x') ? 1 : 0);
      let sp = 0;
      if (perms.includes('s')) sp |= (who.includes('u') ? 4 : 0) |
(who.includes('g') ? 2 : 0);
      if (perms.includes('t')) sp |= 1;
```

```
      ['u', 'g', 'o'].forEach((cls, i) => {
        if (!who.includes(cls)) return;
        if (op === '+') digits[i] |= rwx;
        else if (op === '-') digits[i] &= ~rwx;
        else digits[i] = rwx;
      });
      if (op === '+') special |= sp;
      else if (op === '-') special &= ~sp;
      else {
        const mask = (who.includes('u') ? 4 : 0) | (who.includes('g') ? 2 :
0) | (who.includes('o') ? 1 : 0);
        special = (special & ~mask) | sp;
      }
    }
  }

  const base = parseInt(digits.join(''), 10);
  return { sym: applySpecialBits(octalToSymbolic(base), special), octal:
special * 1000 + base };
}
```

Then replace the whole `if / else if` mode block in the chmod case with:

```
const result = applyChmod(node.permissions, modeArg);
if (!result) {
  return { stdout: '', stderr: `chmod: invalid mode: '${modeArg}'`, exitCode:
1, newCwd: activeCwd, newFs: activeFs, newState: activeState };
}
node.permissions = result.sym;
node.octal = result.octal;
```

3. sudo with a redirect

Evidence. As student, `sudo echo hi > /root/x.txt` creates the file as root. In real bash the shell opens `/root/x.txt` before sudo runs, so it fails with Permission denied. This is a well known LPI trap.

Solution. In `executeLinuxPipeline`, drop the sudo check from the redirect's root test:

```
// before: const isRoot = stage.name === 'sudo' || activeState.isRootOrAdmin
|| ...
const isRoot = activeState.isRootOrAdmin || activeState.currentUser ===
'root';
```

Then add a `tee` command (with `-a`) so the correct answer, `echo hi | sudo tee /root/x.txt`, works. That pairing makes a strong module objective.

4. stderr redirects

Evidence. `echo err 2>> log.txt` writes the text "err 2>" into log.txt: only the first `>` is skipped, so the second becomes a stdout redirect. `echo foo 2> err.txt` prints "foo 2> err.txt". Only `2>/dev/null` and `2>&1` are handled, by regex.

Solution. Pull the stderr redirect out first with a quote aware pass, then run the existing stdout scan on what remains. Delete the `isStderrRedirect` check and the two `.replace()` calls. Tested on 7 inputs, including quoted `"2> x"` and `a2>b`, which correctly stay untouched.

```
export type StderrRedirect = { type: '>' | '>>'; target: string } | { type:
'merge' };

export function extractStderrRedirect(cmd: string): { body: string; stderr?:
StderrRedirect } {
  let inD = false, inS = false;
  for (let i = 0; i < cmd.length; i++) {
    const ch = cmd[i];
    if (ch === '\\' && !inS) { i++; continue; }
    if (ch === '"' && !inS) { inD = !inD; continue; }
    if (ch === "'" && !inD) { inS = !inS; continue; }
    if (inD || inS) continue;
    if (ch === '2' && cmd[i + 1] === '>' && (i === 0 || /\s/.test(cmd[i -
1]))) {
      const m = cmd.slice(i).match(/^2>(>?)\s*(&1|[^\s|&;]+)/);
      if (!m) continue;
      const body = (cmd.slice(0, i) + cmd.slice(i + m[0].length)).trim();
      const stderr: StderrRedirect = m[2] === '&1'
        ? { type: 'merge' }
        : { type: m[1] ? '>>' : '>', target: m[2] };
      return { body, stderr };
```

```
    }
  }
  return { body: cmd };
}
```

Add `stderrRedirect?: StderrRedirect` to `ParsedCommand`. In the engine: `merge` appends stderr to stdout, a `/dev/null` target discards it, and any other target is written with the same `canWriteNode` check stdout uses.

5. mkdir -p

Evidence. `mkdir -p a/b/c` fails with "Permission denied". Two causes: `-p` is ignored (the `hasP` variable was deleted), and `checkPathTraversal` returns false for a missing folder, which `canWriteNode` reports as a permission error.

Solution. In `checkPathTraversal`, treat a missing segment as "not a permission problem" so callers report the real error:

```
if (!current.children || !current.children[seg]) return true; // caller
reports "No such file or directory"
```

Then create each missing parent when `-p` is set:

```
const parents = args.includes('-p');
for (const d of dirs) {
  const targetPath = normalizePath(d, activeCwd, false);
  if (getNodeAtPath(activeFs, targetPath, false)) {
    if (parents) continue;
    return fail(`mkdir: cannot create directory '${d}': File exists`);
  }
  const parts = targetPath.split('/').filter(Boolean);
  const chain = parents ? parts.map((_, i) => '/' + parts.slice(0, i +
1).join('/')) : [targetPath];
  for (const p of chain) {
    if (getNodeAtPath(activeFs, p, false)) continue;
    const check = canWriteNode(activeFs, p, effUser, effGroup, false);
    if (!check.ok) return fail(`mkdir: cannot create directory '${d}':
${check.error}`);
    createNode(activeFs, p, 'directory', '', false, effUser, effGroup, 755);
  }
}
```

(`fail` is the helper from Suggestion 1.) This also fixes `mkdir d` silently succeeding when `d` exists.

6. Lost errors in chains

Evidence. `ls nope; echo after` shows only "after". `lastStderr` is overwritten by every command, so only the final one's error survives.

Solution. Collect them, in both `executeLinuxLine` and the CMD chain loop:

```
const errors: string[] = [];
// inside the loop, after each run:
if (res.stderr) errors.push(res.stderr);
// in the return:
stderr: errors.join('\n'),
```

Real terminals interleave stdout and stderr in order. If you want that later, return a list of `{ stream, text }` chunks instead of two strings.

7. CMD separators and pipes

Evidence. `echo hi & echo there` prints the text literally, because CMD's sequence operator is `&`, not `;`. `ipconfig | findstr IPv4` returns the full ipconfig output, because `executeCmdCommand` never splits on pipes. findstr piping is common on A+ PBQs.

Solution. Give the splitter a shell option. For `cmd`, a single `&` (not `&&`) means sequence and `;` is literal text:

```
export function splitChainedCommandLine(input: string, shell: 'bash' | 'cmd'
= 'bash'): ChainedStage[]
// in the operator checks:
if (shell === 'cmd' && char === '&' && input[i + 1] !== '&') { /* push with
nextOp ';' */ }
if (shell === 'bash' && char === ';') { /* existing */ }
```

For pipes, run `parsePipelinedLine` in the CMD path the way the Linux engine does, and add `findstr` (with `/i` and `/v`) and `more` as stdin filters.

8. rm and mkdir edge cases

Evidence. `rm d` deletes a directory without `-r`. `rm -f nofile` still prints an error.

Solution.

```
const recursive = args.some(a => /^-[a-zA-Z]*[rR]/.test(a));
const force = args.some(a => /^-[a-zA-Z]*f/.test(a));
// per target, before canDeleteNode:
const node = getNodeAtPath(activeFs, targetPath, false);
if (!node) { if (force) continue; return fail(`rm: cannot remove '${t}': No
such file or directory`); }
if (node.type === 'directory' && !recursive) return fail(`rm: cannot remove
'${t}': Is a directory`);
```

9. Supplementary groups

Evidence. `/etc/group` lists `sudo:x:27:student`, but `checkNodePermission` only compares against `currentGroup`. A file owned by group `developers` with student listed as a member would wrongly deny access.

Solution. Read memberships from the virtual `/etc/group` and pass a list:

```
export function getUserGroups(fs: FileNode, user: string, primary: string):
string[] {
  const lines = (getNodeAtPath(fs, '/etc/group', false)?.content ??
'').split('\n');
  const extra = lines.filter(l => (l.split(':')[3] ??
'').split(',').includes(user)).map(l => l.split(':')[0]);
  return [primary, ...extra];
}
// checkNodePermission: (node, user, groups: string[], mode) and test
groups.includes(node.group)
```

The `id` command can use the same helper so `usermod -aG` results show up.

10. Windows redirect mode

Evidence. CMD redirects call `createNode(..., 777)`. It's invisible today, but leaks into `ls -l` style output if the filesystems are ever shared.

Solution. Pass 644, or add a Windows default constant and use it everywhere `createNode` is called with `isWindows = true`.

Suggestions

These aren't bugs, but they would make the code easier to grow and the project stronger as a portfolio piece.

Code structure

1. **Add a result helper.** The full return object `newCwd: activeCwd, newFs: activeFs, newState: activeState` appears 87 times in the Linux engine and 41 in the Windows engine. Two helpers cut most command cases in half and make bugs easier to spot:

   ```
   const ok = (stdout = '') => ({ stdout, stderr: '', exitCode: 0, newCwd:
   activeCwd, newFs: activeFs, newState: activeState });
   const fail = (stderr: string, exitCode = 1) => ({ stdout: '', stderr,
   exitCode, newCwd: activeCwd, newFs: activeFs, newState: activeState });
   ```

2. **Replace the giant switch with a command registry.** A `Record<string, CommandHandler>` with one file per command group (`fileCommands.ts`, `textCommands.ts`, `netCommands.ts`) lets you test each command alone and list available commands for tab completion and the encyclopedia from one source.
3. **Share one chain runner.** The `&&`, `||` and `;` loop is copied in both engines. Move it into `parser.ts` as `runChain(stages, runOne)` so fixes like Finding 6 happen once.
4. **Pick one mutation style.** `executeLinuxPipeline` deep clones the filesystem, then commands mutate nodes in place (`existing.content = ...`). It works, but mixing the two invites bugs where React state changes without a new reference. Either clone once per line and mutate freely, or stay fully immutable.

Testing

5. **Add a regression test for every finding above** so they can't come back. For example, `chmod 4755` gives `rwsr-xr-x`, `sudo echo x > /root/f` fails, `echo e 2> f` writes nothing to stdout, and `mkdir -p a/b/c` creates all three.
6. **Run checks on every push.** A short GitHub Actions workflow running `npm ci`, `npm run lint`, `npm test` and `npm run build` catches problems like yesterday's broken install before anyone clones.

Learning features

7. **Show "why" on errors.** When a student hits Permission denied, show which bit blocked them (for example, "others lack w on /etc"). This turns a dead end into a lesson and reuses `checkNodePermission`.
8. **Save progress.** Completed objectives reset on reload. Storing them in `localStorage` (wrapped in try/catch) is a small change with a big payoff for students.
9. **Add a timed exam mode.** Real PBQs are timed and give no hints. A mode that hides hints and walkthroughs and shows a countdown makes practice closer to test day.

Portfolio polish

10. **Write a real README.** The commit deleted the AI Studio README and nothing replaced it, so the repo page is blank. Add a screenshot, the feature list, the exams covered, how to run it, and a short architecture section (parser, virtual filesystem, two engines).
11. **Remove AI Studio leftovers.** `.env.example` still asks for a `GEMINI_API_KEY` the app no longer uses, `metadata.json` declares a Gemini capability, and the `clean` script deletes a `server.js` that doesn't exist.
12. **Deploy it.** It's a static Vite build, so GitHub Pages or Cloudflare Pages can host it for free. A live link is worth far more to a recruiter than a repo.

Recommended order of work

Fix the exam accuracy bugs first, then clean up the structure before adding features.

- [ ] **Session 1, exam accuracy:** Findings 1, 2 and 3, plus their regression tests. These are the ones that teach wrong answers.
- [ ] **Session 2, shell correctness:** Findings 4, 5, 6 and 8.
- [ ] **Session 3, Windows:** Finding 7 (CMD `&` and findstr pipes), which matters most for A+ PBQs, plus Finding 10.
- [ ] **Session 4, refactor:** Suggestions 1 to 4 (result helper, registry, shared chain runner), done after the fixes so tests protect the refactor. Finding 9 fits here.
- [ ] **Session 5, ship it:** CI, README, AI Studio cleanup and deployment (Suggestions 6, 10, 11, 12).
- [ ] **Later:** Learning features (Suggestions 7, 8, 9).